

Java Full Stack Development

Day 1: Java Quick Start + Agile

Java features and usages	Java is object-oriented, platform independent (write once, run anywhere via JVM), strongly typed, secure and has automatic memory management. Used for enterprise, web, Android and backend systems.
Enterprise App	A large, multi-user business application (banking, insurance, HR) that must be scalable, secure, reliable and integrate with databases and other systems.
Variables and data types	A variable is a named storage location. Primitive types: byte, short, int, long, float, double, char, boolean. Reference types: String, arrays, objects.
Operators	Symbols that perform operations: arithmetic (+ - * / %), relational (== != > <), logical (&& !), assignment (= +=), increment/decrement (++ --).
if / else	Executes a block of code only when a condition is true; else runs when it is false. else-if checks multiple conditions in order.
Loops	Repeat a block of code: for (known count), while (condition checked first), do-while (runs at least once), enhanced for-each (iterate collections/arrays).
Methods	A reusable block of code with a name, parameters and a return type. Avoids duplication and organises logic.
OOP	Object-Oriented Programming models software as objects with state (fields) and behaviour (methods). Four pillars: encapsulation, inheritance, polymorphism, abstraction.
Classes and objects	A class is a blueprint; an object is an instance of that class created with the new keyword.
Constructors	Special method with the class name and no return type, called automatically when an object is created, used to initialise fields. Can be overloaded.
Encapsulation	Keeping fields private and exposing them through public getters/setters, protecting data from direct access.
Inheritance	A class (child) acquires fields and methods of another class (parent) using extends. Promotes reuse (IS-A relationship).
Polymorphism	One name, many forms. Compile-time = method overloading; runtime = method overriding with a parent reference pointing to a child object.
Abstraction	Showing only essential features and hiding implementation details, using abstract classes and interfaces.
Interfaces	A contract declaring methods a class must implement (implements keyword). Supports multiple inheritance of type and loose coupling.
Agile	An iterative software development approach delivering working software in small increments with continuous feedback.
Sprints	Fixed time-boxed iterations (usually 1-4 weeks) in which a planned set of work items is completed and delivered.

Real-time project implementation with Agile	Project work is broken into user stories in a backlog, planned into sprints, built, reviewed and improved sprint by sprint.
Scrum Meeting	Daily stand-up (about 15 minutes) where each member shares what was done yesterday, today's plan and any blockers.
GitHub	Cloud platform for hosting Git repositories: version control, collaboration, branches, commits, push/pull and pull requests.

Day 2: Java Advanced

Collection	The Java Collections Framework provides ready-made data structures (List, Set, Map, Queue) to store and manipulate groups of objects.
ArrayList	Resizable array implementation of List. Ordered, allows duplicates, fast index-based access.
HashMap	Stores key-value pairs. Keys are unique, one null key allowed, no guaranteed order, fast lookup by key.
HashSet	Implementation of Set that stores only unique elements with no guaranteed order.
Comparable	Interface with compareTo() used to define the natural (default) sort order inside the class itself.
Comparator	Interface with compare() used to define custom/multiple sort orders outside the class.
Generics	Type parameters (e.g. List<Customer>) that give compile-time type safety and remove the need for casting.
Exception handling	Mechanism to handle runtime errors so the program does not terminate abruptly. Checked exceptions (compile time) vs unchecked (runtime).
try-catch-finally	try holds risky code, catch handles the exception, finally always executes (used to close resources).
throw	Used to explicitly raise an exception object inside a method.
throws	Declared in a method signature to indicate that the method may pass an exception to its caller.
Custom exceptions	User-defined exception classes (extending Exception or RuntimeException) for business errors, e.g. InsufficientBalanceException.
Java Stream API	Processes collections in a functional style using a pipeline: source, intermediate operations (filter, map, sorted) and terminal operations (collect, forEach, count).
Lambda expressions	Short anonymous functions, (parameters) -> expression, used to implement functional interfaces concisely.
Optional	Container that may or may not hold a value; avoids NullPointerException using methods like isPresent(), orElse(), orElseThrow().

Day 3: Spring Core

Why Spring?	Lightweight framework that simplifies enterprise Java by managing objects, reducing boilerplate and supporting loose coupling, testing and integration.
IoC (Inversion of Control)	Control of object creation and lifecycle moves from the developer to the Spring container.
Dependency Injection	The container supplies (injects) an object's required dependencies instead of the object creating them itself.
Spring Container	The ApplicationContext that creates, configures, wires and manages beans.
Beans	Objects created and managed by the Spring container.
@Component	Generic annotation marking a class as a Spring-managed bean, detected by component scanning.
@Service	Specialised @Component for the business logic layer.
@Repository	Specialised @Component for the data access layer; also translates database exceptions into Spring exceptions.
@Autowired	Tells Spring to inject a matching bean automatically (by type).
Constructor injection	Dependencies are passed through the constructor. Recommended: fields can be final, dependencies are mandatory and code is easy to test.
Configuration	Defining beans using @Configuration classes with @Bean methods, or via annotations and component scanning.
Loose coupling	Classes depend on interfaces rather than concrete implementations, so implementations can change without changing dependent code.

Day 4: Spring Boot

Client-Server Architecture	Client (browser/app) sends a request; server processes it and returns a response over the network.
Role of Application Server	Hosts and runs the web application, handles HTTP requests, threads, sessions and connects requests to application code.
Tomcat	Popular open-source Java web server / servlet container; the default embedded server in Spring Boot.
Spring vs Spring Boot	Spring needs manual configuration and an external server; Spring Boot adds auto-configuration, starters and an embedded server for fast, standalone apps.
Starter dependencies	Pre-packaged dependency sets, e.g. spring-boot-starter-web, spring-boot-starter-data-jpa, that pull in all required libraries.
Auto configuration	Spring Boot configures beans automatically based on the libraries present on the classpath.
Embedded server	The server (Tomcat) is bundled inside the application, so it runs as a simple JAR with java -jar.
application.properties	File for application settings such as server port, database URL, username/password and logging.
Configuration	Customising application behaviour through properties files and @Configuration classes.
Maven	Build and dependency management tool. pom.xml defines dependencies, plugins and build lifecycle (compile, test, package).
Project structure	src/main/java (code: controller, service, repository, entity packages), src/main/resources (properties, static), src/test/java (tests), pom.xml.
REST API development	Building HTTP endpoints that expose application data and operations, typically exchanging JSON.

Day 5: Spring Boot

HTTP	HyperText Transfer Protocol: request-response protocol between client and server, made up of method, URL, headers and body.
REST	Representational State Transfer: architectural style where resources are identified by URLs and handled with standard HTTP methods; stateless.
GET	Retrieves a resource. No change to data.
POST	Creates a new resource.
PUT	Updates/replaces an existing resource.
DELETE	Removes a resource.
HTTP status codes	Indicate the result of a request: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Internal Server Error.
Controller	Class in the web layer that receives HTTP requests, calls the service layer and returns the response.
@RestController	Combines @Controller and @ResponseBody; return values are written directly as JSON.
@RequestMapping	Maps a URL (and optionally a method) to a class or method; commonly used on the class for a base path.
@GetMapping	Maps HTTP GET requests to a method.
@PostMapping	Maps HTTP POST requests to a method.
@PutMapping	Maps HTTP PUT requests to a method.
@DeleteMapping	Maps HTTP DELETE requests to a method.
@RequestBody	Converts the JSON request body into a Java object.
@PathVariable	Reads a value from the URL path, e.g. /customers/{id}.
@RequestParam	Reads a query parameter from the URL, e.g. /customers?city=Chennai.

Day 6: Spring Data JPA

MySQL Setup	Installing MySQL Server and a client (MySQL Workbench) and starting the server.
Creating Database	Creating a schema using CREATE DATABASE dbname; which the application connects to.
JDBC vs JPA	JDBC: manual SQL, connections and result-set mapping. JPA: specification for mapping objects to tables, handling SQL automatically (implemented by Hibernate).
ORM	Object Relational Mapping: mapping Java classes to tables and fields to columns.
Entity	A Java class mapped to a database table.
Primary key	Column that uniquely identifies each row in a table.
@Entity	Marks a class as a JPA entity (table).
@Id	Marks the primary key field.
@GeneratedValue	Generates primary key values automatically, e.g. strategy = GenerationType.IDENTITY (auto increment).
Repository	Data access layer interface that talks to the database.
JpaRepository	Spring Data interface providing built-in methods: save, findById, findAll, deleteById, count, with paging and sorting.
CRUD	Create, Read, Update, Delete: the basic database operations.
Query methods	Queries generated automatically from method names, e.g. findByEmail, findByCityAndStatus.
JPQL basics	Java Persistence Query Language: queries written against entities and fields (not tables) using @Query.
MySQL configuration	Setting spring.datasource.url, username, password and spring.jpa.hibernate.ddl-auto, show-sql in application.properties.

Day 7: Advanced Backend

Entity relationships	Associations between entities that reflect table relationships: one-to-one, one-to-many, many-to-one, many-to-many.
@OneToMany	One entity relates to many of another, e.g. one Customer has many Accounts. Usually mappedBy on this side.
@ManyToOne	Many entities relate to one, e.g. many Accounts belong to one Customer. Owns the foreign key (@JoinColumn).
DTOs	Data Transfer Objects: simple classes carrying only the required data between layers/client, hiding entity internals.
Validation	Checking input data before processing, using Bean Validation (spring-boot-starter-validation).
@Valid	Triggers validation on a @RequestBody object in the controller.
@NotNull	Field must not be null.
@Email	Field must be a valid email format.
@Size	Restricts length of a String or size of a collection (min, max).
Global exception handling	@RestControllerAdvice with @ExceptionHandler methods handles exceptions in one place and returns consistent error responses.
Business logic	Domain rules implemented in the service layer, e.g. minimum balance checks, loan eligibility, account status rules.